

## DASS Assignment 3 Report

# Institute Management System (IMS)

### Team Members

Manik Bansal (Roll Number: 2024101084)

Arnav Agnihotri (Roll Number: 2024101103)

Date: April 20, 2026

## Figma Design Link

<https://www.figma.com/design/wpewNNXoXeVFWHgF0ZYSgP/task3?node-id=0-1&t=UGJnDskPcPs05>

### 1. Project Overview

This submission implements a functional Android prototype of an Institute Management System (IMS) using Jetpack Compose, with role-based workflows and end-to-end flows for selected modules.

The implementation includes:

- Dashboard module (mandatory)
- Time Table module (Category A)
- Student Admission module (Category B)

The prototype is intentionally role-driven:

- Student role focuses on schedule consumption, profile, alerts, and simplified shortcuts.
- Admin role focuses on timetable creation and maintenance, conflict resolution, admission intake and review, and governance controls.

### 2. Scope and Module Selection

Based on the assignment constraints, the implemented end-to-end modules are:

1. Dashboard (mandatory)
  - Student dashboard
  - Admin dashboard
  - Global navigation and role-specific actions
2. Time Table (Category A)
  - Weekly timetable view
  - Admin timetable builder (drag and drop)
  - Conflict alerts and conflict resolution flow
  - Admin timetable publishing visible to student schedule consumption
3. Student Admission (Category B)
  - Multi-step admission form (Personal, Guardian, Academic)
  - Review and submit workflow
  - Admin review and decision (Approve, Pending, Reject)
  - Decision status screen with role-specific return paths

### 3. UX and Visual Design Rationale

The visual direction is a hybrid inspiration from Uber, Claude, and Spotify systems provided in design guidance.

#### 1. Structural discipline from Uber

- Strong hierarchy and compact information layout
- Pill-like call-to-action buttons
- High contrast action affordances
- Task-first arrangement where high-frequency actions are visually promoted and low-frequency actions are visually secondary
- Clear grouping of operational cards to reduce scanning effort in admin-heavy workflows
- Dense-but-readable card patterns optimized for users who must complete actions quickly under routine pressure

Uber-inspired impact in this prototype:

- On dashboard screens, primary decision and action zones are intentionally placed in the first visible viewport so the user can act without deep scrolling.
- In admin views, workflow cards (admissions, timetable, conflicts, review) are arranged as direct operational entry points, mirroring dispatch-style efficiency.
- The interface avoids decorative clutter around critical controls so attention remains on mission-critical tasks and state changes.

#### 2. Editorial warmth from Claude

- Light parchment-like backgrounds
- Serif-heavy section headings
- Gentle card rhythm and approachable academic tone
- Reading-oriented spacing that supports explanatory content, review steps, and status interpretation
- Calm tonal hierarchy that reduces interface intimidation for students in multi-step form journeys
- Human-centered language framing that makes administrative procedures feel understandable rather than mechanical

Claude-inspired impact in this prototype:

- Admission and review flows benefit from a narrative-like progression, where each step feels like a guided checkpoint instead of a raw data-entry sequence.

- Section headings and content blocks are tuned for comprehension, helping users understand “what this section means” before deciding “what to do next”.
- The softer editorial mood is especially useful in student-facing screens, where confidence and clarity are more important than maximal information density.

### 3. Dark utility emphasis from Spotify

- High-contrast dark cards for critical callouts
- Strong content emphasis in high-priority blocks
- Visual escalation strategy where urgent states are immediately distinguishable from routine informational content
- Priority-based color and contrast control to support rapid triage in conflict and decision workflows
- Reduced ambiguity in “act now” scenarios by isolating critical text and actions in dominant visual containers

Spotify-inspired impact in this prototype:

- Critical update regions are intentionally high-contrast so users can detect urgent issues with minimal scan time.
- In conflict and status-oriented modules, dark emphasis areas help separate high-stakes outcomes from general descriptive content.
- This treatment improves visual prioritization on mobile screens where users often glance quickly rather than read linearly.

This hybrid strategy was used to balance:

- Academic seriousness
- Operational clarity
- Accessibility for quick scan and action

Why a hybrid approach was necessary instead of a single-style system:

- A purely utility-first interface could feel efficient for admins but intimidating for students completing admissions forms.
- A purely editorial interface could improve readability but weaken urgency cues needed for conflict and status management.
- A purely dark-emphasis interface could over-signal urgency and reduce information calmness in routine workflows.
- Combining all three allows the UI to adapt tone by task type: operational when needed, explanatory when needed, and urgent when required.

Practical translation across modules:

- Dashboards: Uber-like action hierarchy plus Spotify-like critical emphasis to support rapid orientation.
- Admissions: Claude-like guided readability plus strict validation feedback for accurate completion.
- Timetable and conflicts: Uber-like operational structure plus Spotify-like urgency differentiation for resolution tasks.
- Cross-module continuity: shared components preserve consistency while visual weight shifts based on task criticality.

To convert visual inspiration into concrete implementation choices, the interface design was organized around five practical UI/UX dimensions:

### **A. Layout Hierarchy and Content Rhythm**

- Every major screen follows a vertical rhythm of: Top Bar → Priority Card → Section Label → Action/Information Cards → Bottom Navigation.
- High-priority information (critical updates, decision outcomes, conflicts) is placed above fold to reduce scan latency.
- Related actions are grouped into operational clusters (for example, admissions actions together, timetable actions together) to reduce context switching.
- Cards use consistent padding and corner treatment so users can recognize tappable surfaces without relearning per screen.

### **B. Visual Language and Component Semantics**

- Dark critical cards are reserved for urgency and high attention states, preventing overuse of visual alarm.
- Neutral cards carry routine information, while primary buttons indicate commitment actions such as Save, Submit, Approve.
- Secondary actions (Undo, Back to Review, Edit section) are visually demoted to communicate lower destructive risk.
- Section labels are explicit and task-oriented (for example, “Operational Tasks”, “Active Modules”, “Recent Submissions”) to improve wayfinding.

### **C. Navigation and Discoverability**

- Role-based bottom navigation ensures each user type sees only context-relevant destinations.
- A global search entry point in dashboard top bars supports quick command-style discovery of routes and course detail overlays.
- Search output is query-driven (not prefilled), reducing cognitive noise and making the interface feel responsive to user intent.
- Admin-only capabilities remain reachable within one or two taps from the admin dashboard, minimizing operational delay for frequent governance tasks.

### **D. Interaction Feedback and State Clarity**

- Admission status, conflict state, and resolution outcomes are represented as explicit UI states rather than implicit assumptions.
- Undo actions are present at high-risk interaction points so users can recover from accidental decisions.
- Validation errors are placed near the relevant field or step gate to provide immediate and localizable correction cues.
- Decision-aware status messaging (Approved, Pending, Rejected) keeps users informed without requiring interpretation of raw data.

### **E. Accessibility and Readability Considerations**

- Contrast-heavy priority cards and clear text hierarchy improve readability for quick mobile scanning.
- Touch targets are card- and button-oriented, reducing precision burden compared to compact text links.
- Labels and action names use plain language (for example, “View Schedule”, “Review Admissions”, “Undo Last Status Change”) to reduce ambiguity.
- Forms use step partitioning to reduce memory load and allow progressive completion instead of one long, error-prone page.

Overall, the UI direction prioritizes operational speed, role clarity, and recoverable interaction over ornamental complexity.

## 4. Information Architecture and Role Model

The app uses explicit role-based access and route guards.

1. Student role
  - Home, Schedule, Profile flows
  - No access to admin-only edit operations
  - Read-focused experience for timetable and updates
2. Admin role
  - Home, Alerts, Profile flows
  - Full access to conflict management, timetable builder, and admission governance
3. Access control behavior
  - Restricted route access shows role-denied screen
  - Admin-only screens are isolated from student navigation

This ensures scope correctness and prevents accidental cross-role behavior.

## 5. End-to-End Flow Coverage

### 5.1 Dashboard Flow

1. Login with role switch
2. Role-specific landing dashboard
3. Bottom navigation and contextual actions
4. Global search from top-right icon (student and admin)

Global search behavior:

- No preloaded noisy output
- Typed query drives results
- Supports fuzzy route discovery for Home, Schedule, Profile
- Course discovery for hardcoded Course A to E with details screen

## 5.2 Time Table Flow

Student:

1. Open weekly timetable
2. Filter by day and subject
3. Read daily entries (teacher, room, seat details)
4. View alerts context

Admin:

1. Open conflict alerts
2. Enter conflict resolution screen
3. Apply and undo resolution
4. Open timetable builder
5. Select batch first, then drag-drop courses into grid
6. Save schedule to publish entries

## 5.3 Student Admission Flow

Admin-driven intake and review:

1. Admission personal details
2. Guardian details
3. Academic history
4. Review screen
5. Edit Personal, Guardian, Academic from review
6. Submit
7. Admin review detail
8. Approve, Keep Pending, Reject
9. Undo last status change
10. Decision status screen
11. Approved, Rejected, Pending aware content

Validation behavior:

- Required fields and declarations are hard-blocking
- User cannot proceed to next step with incomplete data



## 6. Usability Principles (From Provided Slides) and Evidence in App

### Principle 1: Consistency

Definition: Similar actions and structures should look and behave consistently across screens.

Where applied:

1. Shared top bar and bottom bar patterns across all major screens.
2. Reusable card, section labels, primary and secondary buttons.
3. Uniform role semantics:
  - Student bottom nav: Home, Schedule, Profile
  - Admin bottom nav: Home, Alerts, Profile
4. Critical Updates dark card style intentionally mirrored on both student and admin dashboards.
5. Consistent naming and destination behavior for common tasks (for example, timetable viewing, profile access, dashboard return path).
6. Repeated status presentation model across modules (pending/approved/rejected and conflict states).

Result:

- Faster learning curve
- Reduced cognitive switching cost
- Predictable navigation behavior
- Lower relearning overhead when moving between modules

Design implication:

- By stabilizing structure and terminology, users spend attention on domain decisions instead of interface interpretation.

### Principle 2: Shortcuts

Definition: Frequent tasks should be reachable quickly without deep navigation.

Where applied:

1. Dashboard shortcuts on student home:

- View Schedule
  - Attendance
  - Apply Leave
2. Admin operational task cards for fast entry:
    - Admissions Intake
    - Timetable Builder
    - Schedule Conflicts
    - Review Admissions
  3. Global search in top-right on both dashboards:
    - Direct route jump (Home, Schedule, Profile)
    - Course lookup by query
  4. Dashboard-first architecture where high-frequency actions are visible without opening side menus or overflow panels.

Result:

- Fewer taps for recurrent tasks
- Better discoverability and memorability of key actions
- Reduced interaction time for admin operational workflows

Design implication:

- The shortcut strategy supports both novice scanning behavior and expert repeat behavior, improving efficiency over time.

### **Principle 3: Error Handling**

Definition: Tell users what is wrong, prevent invalid progression, and provide clear recovery paths.

Where applied:

1. Admission form validation messages at field level.
2. Progress blocking when mandatory fields and declarations are missing.
3. Role denied screens prevent unauthorized operations.
4. Contextual status messaging for conflict and admission actions.
5. Decision flows avoid ambiguous terminal states by explicitly labeling the current outcome.

6. Required checkbox declarations are enforced before submission to prevent invalid form completion.

Result:

- Fewer invalid submissions
- Better user confidence
- Lower chance of silent failure
- Cleaner governance records for admin review decisions

Design implication:

- Preventive validation was prioritized over post-submission correction because admission and scheduling actions are operationally sensitive.

#### **Principle 4: Undo / Reversibility**

Definition: Users should be able to recover from mistakes.

Where applied:

1. Timetable builder:
  - Undo Placement
  - Clear Draft
2. Conflict management:
  - Undo Last Resolve
3. Admission review:
  - Undo Last Status Change
4. Reversible operations are intentionally surfaced near high-impact controls rather than hidden in secondary menus.

Result:

- Safer experimentation
- Lower user anxiety during high-impact edits
- Better trust in admin workflows with reduced fear of irreversible mistakes

Design implication:

- Reversibility supports real-world operational conditions where rapid corrections are often required.

## **Additional UI/UX Heuristic Alignment**

Beyond the four required principles, the implementation also aligns with common interaction heuristics:

1. Visibility of system status
  - Admission and conflict modules expose status outcomes directly in the UI.
  - Decision screens communicate final state and available next action.
2. Match between system and real-world language
  - Terms such as Admissions Intake, Review Admissions, Timetable Builder, and Conflict Alerts mirror institute workflows.
3. User control and freedom
  - Undo affordances and review-stage edit-backs allow non-destructive correction.
4. Recognition rather than recall
  - Action cards and labeled sections reduce memory dependency and improve immediate comprehension.
5. Consistency and standards
  - Shared bars, cards, and button semantics create transferable interaction expectations.

## **7. Technical Architecture Summary**

1. UI stack
  - Native Android with Jetpack Compose
  - Single-activity navigation with composable routes
2. State and data
  - Centralized ViewModel state management
  - Stub repository simulating backend and role data
3. Navigation
  - Route-driven architecture
  - Role checks at route level
  - Single-top behavior to avoid duplicate back stack noise
4. Build and runtime

- Gradle and AGP modern configuration
- JDK 17+ compatible toolchain behavior

## 8. Design Patterns and Anti-Patterns

### 8.1 Patterns used

1. MVVM
  - UI consumes state from ViewModel
  - ViewModel owns business state transitions
2. Reusable component pattern
  - Shared composables for top bar, bottom bar, cards, buttons, labels
3. Role guard pattern
  - Screen-level access controls for admin-only features

### 8.2 Anti-patterns avoided

1. God-screen style duplicated logic
  - Reused components and centralized state instead of copy-paste handlers
2. Silent invalid progression
  - Admission flow blocks transitions until valid input is provided

## 9. Strengths and Weaknesses (Reflection)

Strengths:

1. Strong role separation and route safety.
2. Functional end-to-end flows for timetable and admissions.
3. Usability principles are implemented as behavior, not just visual styling.
4. Practical drag-drop builder with save and undo mechanics.

Weaknesses:

1. Search uses hardcoded course data (A to E) and can be upgraded to repository-backed dynamic indexing.
2. Some interactions rely on static sample content for demo realism.
3. No backend persistence beyond local stub state.

## 10. Mapping to Figma and Final UI

The implemented screens follow the supplied high-fidelity structure:

- Dashboard hierarchy and card rhythm
- Weekly timetable composition and day and subject controls
- Multi-step admissions and review cards
- Admin conflict and builder operational panels

During implementation, iterative refinements were made to align behavior and interaction details with expected flow:

- Role-specific bottom navigation counts
- Search behavior cleanup
- Validation hard blocks
- Decision status transitions
- Critical update consistency across dashboards

## 11. Conclusion

The prototype satisfies the assignment goal of implementing a realistic, role-aware IMS app with a complete dashboard experience and two selected modules end-to-end.

Implemented priorities were:

1. Workflow correctness
2. Usability consistency
3. Operational safety (validation and undo)
4. Fast navigation (shortcuts and global search)

The resulting app demonstrates a coherent functional slice of IMS suitable for viva explanation, flow walkthrough, and further extension into backend integration.